Employee Management System

(Spring boot + Spring security + JWT)

Step 1: Initialize the Spring Boot Project

Dependencies:

- Spring Web
- Spring Boot DevTools
- Spring Data JPA
- Spring Security
- H2 Database
- Lombok (optional but helpful)

Pom.xml

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
</dependency>
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <scope>test</scope>
</dependency>
```

```
        <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt</artifactId>
    <version>0.9.1</version>
</dependency>
<dependency>
    <groupId>javax.xml.bind</groupId>
    <artifactId>jaxb-api</artifactId>
    <version>2.3.1</version>
</dependency>
<dependency>
    <groupId>org.glassfish.jaxb</groupId>
    <artifactId>jaxb-runtime</artifactId>
    <version>2.3.1</version>
</dependency>
    </dependencies>
```

Project Structure

```
- src/
  - main/
    - java/
      - com.example.ems/
        -   config/
            - SecurityConfig.java
        - controller/
            - AuthController.java
            - AuthRequest.java
            - EmployeeController.java
```

- model/
  - Employee.java
  - Role.java
  - User.java
- repository/
  - EmployeeRepository.java
  - UserRepository.java
- security/
  - JwtFilter.java
  - JwtUtil.java
  - JwtProperties.java
- service/
  - CustomUserDetailsService.java
- EmsApplication.java
- resources/
  - application.properties

Step 2: Configure application.properties

```
# H2 Database Configuration
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# JPA Config
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.show-sql=true
```

```
spring.jpa.hibernate.ddl-auto=update

# H2 Console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# JWT Properties
jwt.secret=secret_key
jwt.expiration=3600000
```

Step 3: Create Entity Classes

Role.java – Enum for User Roles

```
package com.example.ems.model;
public enum Role {
   ROLE_USER,
   ROLE_ADMIN
}
```

---

2 User.java – For Authentication

```
package com.example.ems.model;
import jakarta.persistence.*;
import lombok.*;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
```

```
public class User {

    @Id
    private String username;

    private String password;

    @Enumerated(EnumType.STRING)
    private Role role;
}
```

---

3 Employee.java – Main Entity to Manage

```
package com.example.ems.model;
import jakarta.persistence.*;
import lombok.*;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
```

```
    private String department;

    private double salary;
}
```

@Data (from Lombok) generates getters/setters/toString/etc.

Step 4: Create Repository Interfaces

UserRepository.java

```
package com.example.ems.repository;

import com.example.ems.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, String> {
    Optional<User> findByUsername(String username);
}
```

EmployeeRepository.java

```
package com.example.ems.repository;

import com.example.ems.model.Employee;
import org.springframework.data.jpa.repository.JpaRepository;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
}
```

Step 5: JWT & Security Configuration

JwtUtil.java – Generate & Validate Tokens

```
package com.example.ems.security;
import java.util.Date;
import org.springframework.stereotype.Component;
import io.jsonwebtoken.*;

@Component
public class JwtUtil {
   private final JwtProperties jwtProperties;

   public JwtUtil(JwtProperties jwtProperties) {
      this.jwtProperties = jwtProperties;
   }
   public String generateToken(String username) {

      try {
         String token = Jwts.builder()
               .setSubject(username)
               .setIssuedAt(new Date())
               .setExpiration(new Date(System.currentTimeMillis() +
jwtProperties.getExpiration()))
               .signWith(SignatureAlgorithm.HS256, jwtProperties.getSecret())
               .compact();
```

```
        return token;
    } catch (Exception e) {
        e.printStackTrace();
        return null;
    }
}

public String extractUsername(String token) {
    try {

        return Jwts.parser()
                .setSigningKey(jwtProperties.getSecret())
                .parseClaimsJws(token)
                .getBody()
                .getSubject();

    } catch (JwtException e) {

        throw new IllegalArgumentException("Invalid JWT token");
    }

}

public boolean validateToken(String token) {
    try {
        Jwts.parser().setSigningKey(jwtProperties.getSecret()).parseClaimsJws(token);
        return true;
```

```
        } catch (JwtException e) {
            return false;
        }
    }
}
```

JwtProperties.java

```
package com.example.ems.security;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
@Component
@ConfigurationProperties(prefix = "jwt")
public class JwtProperties {
    private String secret;
    private long expiration;
}
```

JwtFilter.java – Intercept Requests & Authenticate

```
package com.example.ems.security;
```

```
import com.example.ems.service.CustomUserDetailsService;
import org.springframework.beans.factory.annotation.Autowired;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToke
n;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

@Component
public class JwtFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtil jwtUtil;

    @Autowired
    private CustomUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse
response, FilterChain chain)
            throws ServletException, IOException {
```

```
        String authHeader = request.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7);
            String username = jwtUtil.extractUsername(token);

            if (username != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {
                UserDetails userDetails =
userDetailsService.loadUserByUsername(username);

                if (jwtUtil.validateToken(token)) {
                    UsernamePasswordAuthenticationToken auth =
                            new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());
                    SecurityContextHolder.getContext().setAuthentication(auth);
                }
            }
        }
        chain.doFilter(request, response);
    }
}
```

Step 6: Authentication & Security Configuration

CustomUserDetailsService.java

This connects your app's User model to Spring Security's UserDetails system

```
package com.example.ems.service;

import com.example.ems.model.User;

import com.example.ems.repository.UserRepository;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.security.core.authority.SimpleGrantedAuthority;

import org.springframework.security.core.userdetails.*;

import org.springframework.stereotype.Service;


import java.util.Collections;


@Service

public class CustomUserDetailsService implements UserDetailsService {


    @Autowired

    private UserRepository userRepository;


    @Override

    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {

        User user = userRepository.findByUsername(username)

                .orElseThrow(() -> new UsernameNotFoundException("User not found"));


        return new org.springframework.security.core.userdetails.User(

                user.getUsername(),

                user.getPassword(),
```

```
            Collections.singleton(new SimpleGrantedAuthority(user.getRole().name()))
        );
    }
}
```

SecurityConfig.java

```
package com.example.ems.config;

import com.example.ems.security.JwtFilter;

import com.example.ems.service.CustomUserDetailsService;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;

import org.springframework.security.authentication.*;

import
org.springframework.security.config.annotation.authentication.configuration.Authen
ticationConfiguration;

import org.springframework.security.config.annotation.web.builders.HttpSecurity;

import org.springframework.security.config.http.SessionCreationPolicy;

import org.springframework.security.crypto.bcrypt.*;

import org.springframework.security.web.SecurityFilterChain;

import
org.springframework.security.web.authentication.UsernamePasswordAuthentication
Filter;


@Configuration

public class SecurityConfig {
```

```
    @Autowired
    private JwtFilter jwtFilter;

    @Autowired
    private CustomUserDetailsService userDetailsService;

    @Bean
    public AuthenticationManager
authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public BCryptPasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
                .csrf(csrf -> csrf.disable())
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/auth/**").permitAll()
                        .anyRequest().authenticated()
                )
                .sessionManagement(sess ->
sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .userDetailsService(userDetailsService)
```

```
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}
```

What this config does:

- Disables CSRF (for APIs)
- Allows /auth/** (login/register) without authentication
- Secures all other endpoints
- Adds JWT filter **before** Spring's own login filter
- Makes sessions stateless
- Sets custom user detail logic and password encoder

Step 7: Create Authentication Controller (/auth/login & /auth/register)

- Register (with username & password)
- Log in (and receive a JWT)

AuthController.java

```
package com.example.ems.controller;

import com.example.ems.model.RoleModel;
import com.example.ems.model.UserModel;
import com.example.ems.repository.UserRepository;
import com.example.ems.security.JwtUtil;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.*;
```

```
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/auth")
public class AuthController {

    @Autowired
    private AuthenticationManager authenticationManager;

    @Autowired
    private JwtUtil jwtUtil;

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    // 🔓 Login Endpoint
    @PostMapping("/login")
    public ResponseEntity<String> login(@RequestBody AuthRequest request) {
        // Authentication authentication = authenticationManager.authenticate(
        // new UsernamePasswordAuthenticationToken(request.getUsername(),
        // request.getPassword()));

        // UserDetails user = (UserDetails) authentication.getPrincipal();
        // return jwtUtil.generateToken(user.getUsername());

        try {

            if (request.getUsername() == null || request.getPassword() == null) {

                return ResponseEntity.badRequest().body("Username or password is
missing.");
            }
            Authentication authentication = authenticationManager.authenticate(
```

```
                new
UsernamePasswordAuthenticationToken(request.getUsername(),
request.getPassword()));

            UserDetails user = (UserDetails) authentication.getPrincipal();
            String token = jwtUtil.generateToken(user.getUsername());

            return ResponseEntity.ok(token);
        } catch (BadCredentialsException e) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid
username or password");
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                    .body("Something went wrong: " + e.getClass().getSimpleName() +
" - " + e.getMessage());
        }
        // } catch (AuthenticationException e) {
        // return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid
username
        // or password");
        // }
    }

    // 📝 Register Endpoint
    @PostMapping("/register")
    public String register(@RequestBody AuthRequest request) {
        if (userRepository.existsById(request.getUsername())) {
            return "User already exists!";
        }

        UserModel user = new UserModel();
        user.setUsername(request.getUsername());
        user.setPassword(passwordEncoder.encode(request.getPassword()));
        user.setRole(RoleModel.ROLE_USER); // Default role

        userRepository.save(user);
        return "User registered successfully!";
    }
}
```

AuthRequest.java

```
package com.example.ems.controller;

import lombok.Data;

@Data
public class AuthRequest {
   private String username;
   private String password;
}
```

Step 8: Employee CRUD Controller (protected by JWT)

Create the EmployeeController — this will let authenticated users create, read, update, and delete employees.

```
package com.example.ems.controller;

import com.example.ems.model.Employee;

import com.example.ems.repository.EmployeeRepository;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.web.bind.annotation.*;


import java.util.List;

import java.util.Optional;


@RestController

@RequestMapping("/employees")

public class EmployeeController {


   @Autowired
```

```
    private EmployeeRepository employeeRepository;

    // 🔍 Get all employees
    @GetMapping
    public List<Employee> getAllEmployees() {
        return employeeRepository.findAll();
    }

    // 🔍 Get employee by ID
    @GetMapping("/{id}")
    public Optional<Employee> getEmployeeById(@PathVariable Long id) {
        return employeeRepository.findById(id);
    }

    // ➕ Create new employee
    @PostMapping
    public Employee createEmployee(@RequestBody Employee employee) {
        return employeeRepository.save(employee);
    }

    // 🖍️ Update existing employee
    @PutMapping("/{id}")
    public Employee updateEmployee(@PathVariable Long id, @RequestBody
Employee employeeDetails) {
        Employee emp = employeeRepository.findById(id).orElseThrow();

        emp.setName(employeeDetails.getName());
        emp.setDepartment(employeeDetails.getDepartment());
```

```
        emp.setSalary(employeeDetails.getSalary());

        return employeeRepository.save(emp);
    }

    // ❌ Delete employee
    @DeleteMapping("/{id}")
    public String deleteEmployee(@PathVariable Long id) {
        employeeRepository.deleteById(id);
        return "Employee deleted";
    }
}
```

Postman Testing

Register

Request Curl:-

```
curl --location 'localhost:8080/auth/register' \
--header 'Content-Type:  application/json' \
--data '{
  "username": "johnh",
  "password": "1234"
}
'
```

Response:-

User already exists!

Login

Request Curl:-

curl --location 'localhost:8080/auth/login' \

--header 'Content-Type: application/json' \

--data '{

  "username": "johnh",

  "password": "1234"

}

'Response:-

Generate token - eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJqb2huaCIsImlhdCI6MTc0NTE0MTE1NSwiZXhwIjoxNzQ1MTQ0NzU1fQ.jigAiQ25Sg1oq-mWyD3ZP2Cqtzqy66BsCxW3ZlltJAI

Employee Post method

Request Curl:-

curl --location 'http://localhost:8080/employees' \

--header 'Content-Type: application/json' \

--header 'Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJqb2huaCIsImlhdCI6MTc0NTE0MTE1NSwiZXhwIjoxNzQ1MTQ0NzU1fQ.jigAiQ25Sg1oq-mWyD3ZP2Cqtzqy66BsCxW3ZlltJAI' \

--data '{

    "name": "Alice",

    "department": "HR",

    "salary": 50000

}'

Response:-

{

    "id": 1,

    "name": "Alice",

```
    "department": "HR",
    "salary": 50000.0
}
```


Employees GET method

Request Curl:-


```
curl --location 'http://localhost:8080/employees' \
--header 'Content-Type: application/json' \
--header 'Authorization:  Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJqb2huaCIsImlhdCI6MTc0NTE0MTE1NSwiZXhwIjoxNzQ1MTQ0NzU1fQ.jigAiQ25Sg1oq-mWyD3ZP2Cqtzqy66BsCxW3ZlltJAI' \
--data '''
```


Response:-


```
[
    {
        "id": 1,
        "name": "Alice",
        "department": "HR",
        "salary": 50000.0
    }
]
```


Employees PUT method

Request Curl:-


```
curl --location --request PUT 'http://localhost:8080/employees/1' \
--header 'Content-Type: application/json' \
--header 'Authorization:  Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJqb2huaCIsImlhdCI6MTc0NTE0MTE1NSwiZXhwIjoxNzQ1MTQ0NzU1fQ.jigAiQ25Sg1oq-mWyD3ZP2Cqtzqy66BsCxW3ZlltJAI' \
--data '{
  "name": "Alice Cooper",
```

"department": "Finance",

"salary": 60000

}

'

Response:-

{

"id": 1,

"name": "Alice Cooper",

"department": "Finance",

"salary": 60000.0

}

Employees DELETEmethod

Request Curl:-

curl --location --request DELETE 'http://localhost:8080/employees/1' \

--header 'Content-Type: application/json' \

--header 'Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJqb2huaCIsImlhdCI6MTc0NTE0MTE1NSwiZXhwIjoxNzQ1MTQ0NzU1fQ.jigAiQ25Sg1oq-mWyD3ZP2Cqtzqy66BsCxW3ZlltJAI''

Response:-

Employee deleted